PS 3 · LegalTech | Post-quantum | Zero-trust | AWS + Ethereum Sepolia

Sakshya

Tamper-evident, quantum-safe, zero-trust judicial & forensic document management

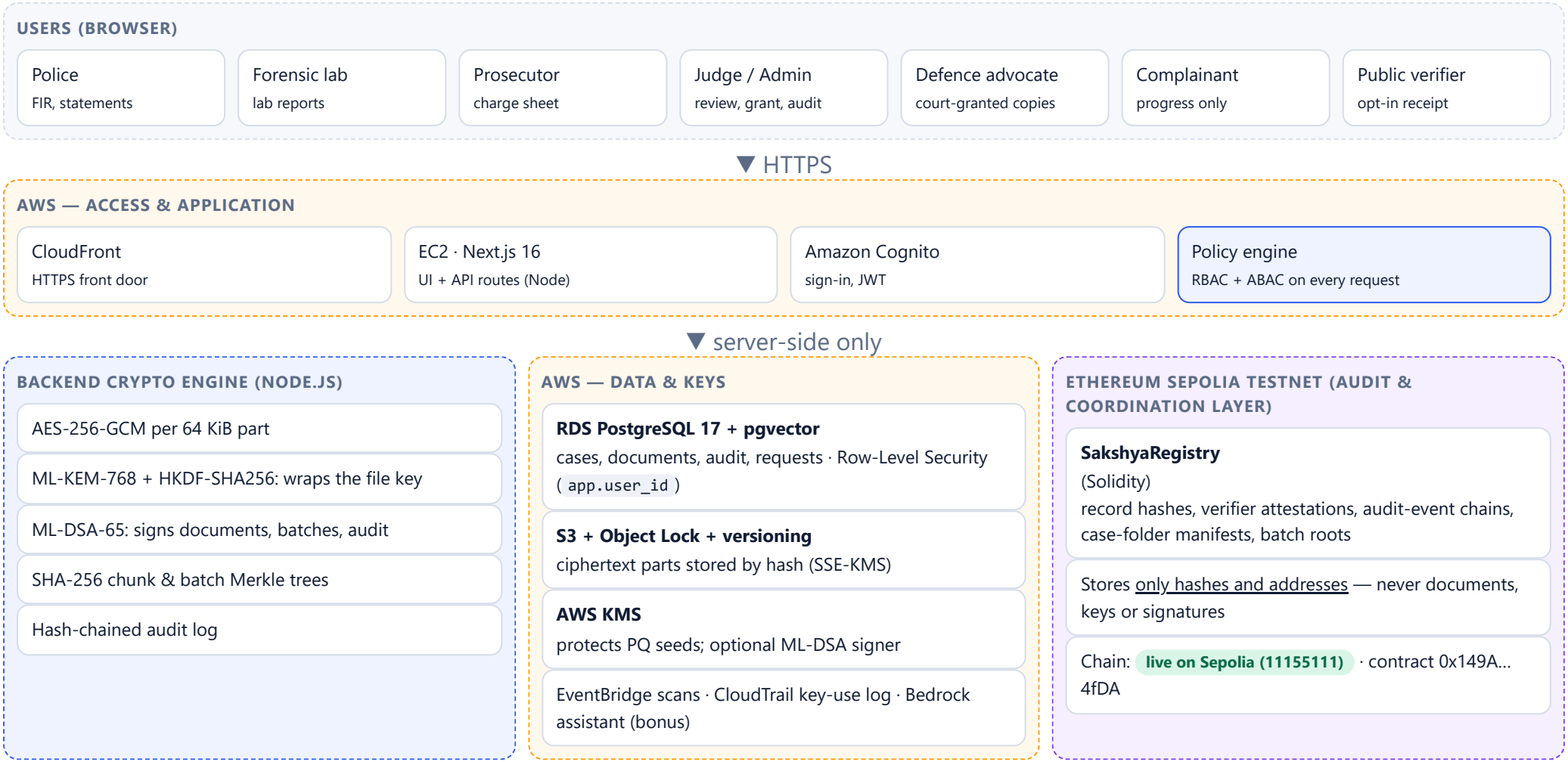
System architecture · workflows · access model · trust boundaries
Hack-a-Night 2026 · PVGCOE Nashik · prepared 9 Oct 2026

How to read this document

p.2 System architecture · p.3 Evidence upload workflow · p.4 Verification & tamper detection · p.5 Case folder, parts & assembly · p.6 Court request workflow · p.7 Roles & access · p.8 Data model · p.9 Trust boundaries & honest status

1 · System architecture

Three lanes: who can see what, where the evidence lives encrypted, and what is committed publicly.



Key idea: The chain is the notary, not the vault. Encryption and post-quantum cryptography run in our backend; the chain publicly timestamps hashes so later edits are provable.

2 · Evidence upload workflow

From a police officer's upload to a sealed, signed, audited and chain-committed record.

1. **Authenticate & authorise**
   Cognito JWT verified. Policy engine checks role, case assignment and the role→document-type matrix. RDS row-level security repeats the check in the database. A refusal is logged as **DENIED**.

2. **Validate**
   ≤ 4 MiB; pdf / docx / txt / png / jpg; server recomputes everything — nothing from the client is trusted.

3. **Fingerprint plaintext**
   SHA-256 of the file; 4 KiB chunk Merkle tree → `chunkRoot` + chunk leaf list.

4. **Encrypt in parts**
   Random 256-bit key (DEK). File split into 64 KiB parts; each part AES-256-GCM encrypted (AAD = docId|index|count). Part address = SHA-256 of the stored bytes.

5. **Wrap the key (post-quantum)**
   ML-KEM-768 encapsulation → HKDF-SHA256 → key-encryption key → AES-GCM wraps the DEK. Only the wrapped key is stored.

6. **Sign**
   Uploader's ML-DSA-65 key signs the canonical JSON {docId, caseId, type, version, sha256, chunkRoot, partRoot, timestamp}.

7. **Store**
   Ciphertext parts → S3 (by hash). Metadata, wrapped key, signature → RDS (append-only). Plaintext never touches disk or DB.

8. **Record**
   Audit entry (with **legal basis**) hash-chained; case manifest version +1; record registered on the chain registry.

Receipt shown to the user
plaintext SHA-256 · AES-256-GCM ✓ · ML-KEM-768 ✓ · ML-DSA-65 signature ✓ · audit entry ID · status *UNANCHORED → PENDING_ANCHOR → VERIFIED*

What is stored where

Item	Where
Encrypted parts	S3 (key = SHA-256 of part)
Wrapped file key, IVs	RDS
Hashes, chunk leaves, signature	RDS
Audit chain	RDS + hash on chain
Document / manifest / batch hashes	Sepolia registry
Plaintext, private keys	**Nowhere** (memory only)

Corrections never overwrite: a new version is appended with `parentId`, and the registry enforces `version > parent.version`.

3 · Verification & tamper detection

Runs on every view, on demand, and in integrity scans. Six independent checks; green only when all pass.

① Parts present & unaltered
Fetch each part; SHA-256(bytes) must equal its address. An edited part is named without decrypting: "*part 2 altered*".

② Authenticated decryption
Unwrap key (ML-KEM-768), AES-256-GCM decrypt each part. Swapped, reordered or cross-document parts fail.

③ Plaintext SHA-256
Recompute and compare with the signed original.

④ Chunk Merkle root
Diff 4 KiB chunk hashes → **exact chunk + byte range**, e.g. *chunk 2, bytes 8192–12287*.

⑤ ML-DSA-65 signature
Verified off-chain by our backend verifier; verdict + evidence hash attested on chain.

⑥ Batch proof & on-chain root
Document leaf ∈ batch root; batch root equals the root stored on the chain.

Insider-attack scenario (demo centrepiece)

Attack
Insider with key access decrypts, flips one plaintext byte (offset 9000), re-encrypts validly, overwrites the stored object.

▶

Why decryption passes
GCM is valid for the new bytes, so "can it be opened?" says yes. This is why decryption alone is not enough.

▶

Detection
SHA-256 ≠ signed hash · chunk root ≠ original · part address ≠ stored. **Chunk 2, bytes 8192–12287**. Last accessors listed (context, not proof of guilt).

▶

Recovery
S3 Object Lock kept the original version. Authorised restore appends a new signed version; the damaged one stays on record in red.

Status	Meaning	Colour
VERIFIED	all six checks pass	**green**
TAMPERED	a content / hash / part check failed; ranges reported	**red**
PENDING_ANCHOR	content checks pass; batch not yet confirmed on chain	**amber**
UNANCHORED / ERROR	not committed yet / could not verify — never shown as green	grey / red

4 · Case folder, parts & court assembly

One FIR = one case folder. Every document is stored as encrypted parts; the court rebuilds and verifies the whole folder.

Case folder (FIR NSK/2026/0142)

FIR	Witness statement	Forensic report
police	police	lab

▼ each document

Document → parts

part 0 · a41f… | part 1 · 7c02… | part 2 · e9b8… | part 3 · 03dd…

64 KiB each · AES-256-GCM · filed under its own SHA-256

▼ leaves

Case manifest

Merkle root over every document leaf (id + file hash + chunk root + part root). Each new document seals

manifest version n+1

on the chain; versions only grow.

"Assemble case folder" (court / prosecutor)

1. Policy check; advocates only get documents granted to them.
2. Fetch every part by hash; verify address.
3. Decrypt, rebuild each document, verify hash / chunks / signature.
4. Recompute the manifest root; compare with the on-chain root.
5. Live progress "
   38/42 parts verified
   " → ZIP bundle (files + manifest.json + hash certificate) only if everything is VERIFIED.
   Audited as FOLDER_ASSEMBLED.

Splitting per document (not the whole folder as one blob) keeps access granular and lets new evidence be added later.

5 · Court request workflow & complainant progress

Advocates obtain documents only through a judge's order; everything — who asked, who decided, who looked — is recorded with its legal basis.

1 · Advocate files
documents wanted, legal basis + citation, reason → *REQUEST_FILED*

2 · Judge reviews
sees the request, case and documents

3 · Decision
Grant (all or subset, optional expiry) with **order reference + note**, or Reject → *REQUEST_GRANTED / REJECTED*

4 · Access
View + **watermarked** download (advocate, order ref, hash, time); every access audited with *COURT_ORDER*

5 · Expiry
Access ends at expiry; the full trail remains

Every audit entry records

who (pseudonymous on chain) · what action · which document / case · when · **legal basis** {code, citation, order ref} · previous hash → entry hash (chain). The legal basis is inside the hash, so the reason cannot be edited later.

Legal basis codes

INVESTIGATION · FORENSIC_ASSIGNMENT · PROSECUTION_ASSIGNMENT · JUDICIAL_AUTHORITY · COURT_ORDER · STATUTORY_DISCLOSURE · VICTIM_INFORMATION · SYSTEM_INTEGRITY

Citations are entered by the judge / advocate and stored verbatim. The system records the basis; it does not interpret law. Statute section numbers should be confirmed with legal counsel before being quoted.

Complainant progress view

FIR registered · Investigation · Forensic analysis · Charge sheet · Trial · Judgment

Milestones are added by the investigating officer or the court (audited). The complainant sees dates and public-safe notes only — no documents, hashes of evidence, or other parties.

6 · Roles & access control (RBAC + ABAC)

Enforced twice: in the API policy layer and again inside the database by row-level security. Denials are logged.

Role	Can upload	Can read	Other powers	Cannot
POLICE	FIR, witness statements (own cases)	own cases	create case, add milestones	edit/delete, other stations' cases
FORENSIC_LAB	forensic reports (assigned)	assigned cases	—	unassigned cases (DENIED + logged)
PUBLIC_PROSECUTOR	charge sheets (assigned)	full assigned case file	assemble folder	edit evidence
JUDGE	—	all cases (read-only)	grant/reject requests, run scans, audit views	modify evidence
ADMIN	—	all cases (read-only)	audit chain, scans, batches	modify evidence, decide requests
DEFENCE_ADVOCATE	—	**only granted documents**, until expiry	file document requests	anything without a court order
COMPLAINANT	—	own case **milestones only**	—	see documents or hashes

ABAC attributes

- case assignment
- police station / jurisdiction
- document type
- active court grant + expiry

Append-only everywhere: no edit or delete in the UI, API or database (triggers block UPDATE/DELETE on documents and audit; the chain registry is write-once per record). Zero-trust: every request is re-authenticated and re-authorised.

7 · Data model (RDS PostgreSQL)

Core tables; `FORCE ROW LEVEL SECURITY` with policies keyed on `current_setting('app.user_id')`.

users
id (Cognito sub) · name · role · station · clearance · ML-DSA public key hash

cases
id · fir_no · station · stage · status · created_by

case_assignments
case_id · user_id · role

milestones
case_id · stage · note · recorded_by · at

documents
id · case_id · type · version · parent_id · sha256 · chunk_root · chunk_leaves · part_root · wrapped_dek · signature · batch_id · merkle_proof

document_parts
doc_id · index · address (sha256) · size

case_manifests
case_id · version · root · doc_count · tx

merkle_batches
id · root · signature · tx_hash · status

document_requests
id · case_id · requester · documents · legal_basis · status

request_grants
request_id · document_id · order_ref · expires_at

audit_log
id · actor · action · doc/case · legal_basis · prev_hash · entry_hash · at

doc_embeddings
(bonus) chunk text · vector(1024) · case_id

On-chain (SakshyaRegistry)

registerRecord
(write-once, versions)

attestSignature
(off-chain verdict + evidence hash)

logAuditEvent
(per-document / per-case hash chain)

sealCaseManifest
(folder versions 1,2,3…)

anchorBatch
(Merkle batch roots)

Roles on the contract: RECORDER (backend), VERIFIER (off-chain ML-DSA verifier), PAUSER/ADMIN. Pseudonymous actor hashes only; no plaintext, keys or signatures on chain.

8 · Trust boundaries & honest status

What the system proves — and what it does not

Claim	Basis
A record existed unchanged at time T	hash committed on chain at T
Content was altered (and where)	hash / chunk / part mismatch
Who accessed it, and why	hash-chained audit with legal basis
The ML-DSA signature is valid	**Our backend verifier** checked it; the chain stores its attestation. The blockchain does not verify post-quantum signatures. Anyone can re-verify from the stored signature and key hashes.
Recent accessors are guilty	**Not claimed** — context only

Post-quantum scope: ML-KEM-768 protects document-key wrapping and ML-DSA-65 signs commitments. The chain's own consensus and transaction signatures are not quantum-safe. Trust dependencies: the backend verifier and the protection of signing / decryption keys (KMS).

Build status (10 Oct 2026)

Component	Status
Crypto: ML-KEM-768 / ML-DSA-65 / AES-256-GCM, 64 KiB encrypted parts	built · tested
Merkle trees, byte-range tamper localisation	built · tested
Content-addressed store on S3 (Object Lock + versioning)	live
PostgreSQL 17 on RDS: row-level security, append-only tables, hash-chained audit log and FIR register	live
API: cases, documents, verify, restore, audit, scans, requests, assembly, chain feed	live · 133 tests + live end-to-end run
SakshyaRegistry on Ethereum Sepolia; batch anchoring; on-chain root check	live
Cognito sign-in, EC2 + Caddy HTTPS, KMS	live
40 fictional case folders (148 documents)	imported
Bedrock assistant, public verify receipts, EventBridge scans, CloudFront	not built

"Live" = running at https://3-109-114-12.sslip.io. The chain is a public testnet: it stores hashes and the verifier's attestation; ML-DSA signatures are verified off-chain by our backend. Court-granted PDF/Word/image copies are marked by file name and header only; plain-text copies are stamped inside the file.